

A++

Plain-English Programming Language

Comprehensive User Manual

"Code in the language you already speak."

Version 0.9 Prototype

Author: Vishnu Theertha. A

Location: Bengaluru, Karnataka, India

Started: 21 / 04 / 2026 · Updated: 23 / 04 / 2026

MIT License · Open Source

1. Introduction to A++

A++ (pronounced *A-plus-plus*) is an open-source, plain-English programming language designed to let anyone write working programs using natural sentences — no curly braces, no semicolons, no cryptic symbols. Every A++ program is transpiled into clean Python and executed at full speed, giving beginners a comfortable entry point and experienced programmers a readable scripting alternative.

1.1 Design Philosophy

- **Readability first** — Every statement must read like an English sentence.
- **Forgiveness** — The IECE engine corrects typos, fills missing keywords, and learns from your habits so the language meets you where you are.
- **Zero required symbols** — Operators like **is greater than** and **>** are always interchangeable. Use whichever feels natural.
- **Full Python power** — Under the hood A++ generates Python, giving you access to the entire Python ecosystem.

1.2 How to Run A++

A++ requires Python 3.10 or later and no third-party packages. The interpreter is a single file: [applus.py](#).

```
# Start the interactive REPL

python3 applus.py

# Run a .applus file

python3 applus.py myprogram.applus

# Run with full traceback on errors

python3 applus.py myprogram.applus --debug
```

■ **Tip:** When you start the REPL, you see a numbered prompt like `A++[1]>`. Type any A++ statement and press Enter to run it.

2. Data Types

A++ supports the same fundamental types as Python. You never need to declare a type — it is inferred automatically from the value you assign.

Type	A++ Example	Description
<code>int</code>	set x to 42	Whole number (positive or negative)
<code>float</code>	set pi to 3.14159	Decimal number
<code>string</code>	set name to "Anand"	Text, enclosed in double quotes
<code>boolean</code>	set flag to true	true or false (also: nothing = None)
<code>list</code>	set nums to [1, 2, 3]	Ordered, mutable sequence
<code>map</code>	set person to map	Key-value dictionary
<code>tuple</code>	set t to (1, 2, 3)	Ordered, immutable sequence
<code>nothing</code>	set x to nothing	Absence of value (Python None)

2.1 String Interpolation

Any string containing `{variable}` is automatically converted to a Python f-string. You never need to write f-strings manually.

```
set name to "Anand"

set score to 95

say "Hello, {name}! Your score is {score}."

// Output: Hello, Anand! Your score is 95.
```

2.2 Type Conversion (Phase 3)

Convert between types with the `convert` keyword:

```
set age_text to "25"

set age to convert age_text to int

set pi_str to "3.14"
```

```
set pi      to convert pi_str to float
```

```
set flag    to convert 1 to boolean
```

```
set as_text to convert 42 to string
```

Target Type	Keyword	Example
Integer	int / integer	convert x to int
Float	float / number	convert x to float
String	string / text	convert x to string
Boolean	boolean / bool	convert x to boolean
List	list	convert x to list

3. Variables and Assignment

3.1 Basic Assignment

All variables are created with `set VAR to VALUE`. Variable names are case-sensitive and can contain letters, digits, and underscores (but must start with a letter or underscore).

```
set x to 10

set name to "Anand"

set pi to 3.14159

set flag to true

set scores to [85, 92, 78]

set person to map
```

3.2 Augmented Assignment

Modify an existing variable with English or symbolic augmented assignment:

```
increase score by 10      // score += 10

decrease health by 1      // health -= 1

add 5 to total            // total += 5

subtract 3 from balance   // balance -= 3

multiply price by 1.1     // price *= 1.1

divide result by 4        // result /= 4
```

3.3 Type-Annotated Assignment (Optional)

You may optionally annotate the type for documentation. It has no runtime effect — it is a human readability aid.

```
set age as number to 25
```

```
set name as string to "Anand"
```

3.4 Abbreviations Accepted by IECE

IECE recognises JavaScript/Python-style assignment forms and silently translates them:

You type	A++ understands
<code>var x = 10</code>	set x to 10
<code>let name = "A"</code>	set name to "A"
<code>const PI = 3.14</code>	set PI to 3.14

4. Operators

Every operator works in **both English and symbol form**. You can mix them freely within the same program.

4.1 Relational Operators

English Form	Symbol	Meaning
is equal to	<code>==</code>	Equal
is not equal to	<code>!=</code>	Not equal
is greater than	<code>></code>	Greater than
is less than	<code><</code>	Less than
is greater than or equal to	<code>>=</code>	Greater than or equal
is less than or equal to	<code><=</code>	Less than or equal

4.2 Arithmetic Operators

English Form	Symbol	Meaning
plus	<code>+</code>	Addition
minus	<code>-</code>	Subtraction
divided by	<code>/</code>	Division
modulo / mod	<code>%</code>	Remainder
to the power of	<code>**</code>	Exponentiation
floor divided by	<code>//</code>	Integer division

4.3 Logical Operators

English Form	Symbol	Meaning
and / and also	<code>&&</code>	Logical AND
or / or else	<code> </code>	Logical OR
not	<code>!</code>	Logical NOT

4.4 Membership Operators

A++ Form	Python Equivalent
<code>x is in list</code>	<code>x in list</code>
<code>x is not in list</code>	<code>x not in list</code>

4.5 Boolean Literals

A++ Keyword	Python Value
<code>true</code>	<code>True</code>
<code>false</code>	<code>False</code>
<code>nothing / null</code>	<code>None</code>
<code>is true</code>	<code>== True</code>
<code>is false</code>	<code>== False</code>
<code>is nothing</code>	<code>is None</code>

```

set x to 10

set y to 3


set sum    to x plus y           // 13

set diff   to x minus y         // 7

set power  to x to the power of y // 1000

set check  to x is greater than y // True

set member to 5 is in [1, 3, 5, 7] // True

```

5. Control Flow

A++ supports two equally valid block styles: **Classic style** (no colon, closed with `end if / end loop`) and the newer **Colon style** (colon after the condition, closed with `end of if-else / end of for / end of while`). Both are fully supported and can be mixed.

5.1 If / Else If / Else

■ Colon Style (new)	■ Classic Style
<pre>set grade to 85 if grade >= 90 then: say "Excellent!" else if grade >= 75 then: say "Good work!" else: say "Keep going!" end of if-else</pre>	<pre>set grade to 85 if grade >= 90 then say "Excellent!" else if grade >= 75 then say "Good work!" else say "Keep going!" end if</pre>

■ **Tip:** `end of if-else` closes any if/else-if/else chain regardless of how many branches it has. The older `end if` still works — IECE auto-corrects it.

5.2 While Loop

■ Colon Style	■ Classic Style
<pre>set x to 1 while x <= 5: say x increase x by 1 end of while</pre>	<pre>set x to 1 while x <= 5 say x increase x by 1 end loop</pre>

5.3 Repeat Loop

Run a block a fixed number of times. The colon is optional.

```
repeat 5 times

  say "hello"

end loop


// Colon form also works:

repeat 5 times:

  say "hello"

end loop
```

5.4 For-Each Loop

Iterate over any list, range, or sequence:

```
set fruits to ["apple", "mango", "grape"]


for each fruit in fruits:

  say fruit

end of for


// Using built-in range:

for each i in range(1, 6):

  say i
```

```
end of for
```

```
// range with step (even numbers 0-8):
```

```
for each n in range(0, 10, 2):
```

```
    say n
```

```
end of for
```

range() Form	Equivalent	Description
<code>range(5)</code>	<code>range(0, 5)</code>	0 to 4
<code>range(1, 6)</code>	<code>range(1, 6)</code>	1 to 5
<code>range(0, 10, 2)</code>	<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8

6. Functions

6.1 Defining and Calling Functions

```
// No parameters

define greet

    say "Hello!"

end function


// One parameter

define greet taking name

    say "Hello, {name}!"

end function


// Multiple parameters

define add taking a, b

    return a + b

end function


// Calling functions

call greet

call greet with "Anand"
```

```
set result to add(10, 32)      // returns 42
```

6.2 Return Values

```
define circle_area taking radius

    set pi to 3.14159

    return pi * radius * radius

end function

set area to circle_area(7)

say "Area = {area}"
```

■ **Tip:** Use `return` anywhere inside a function. If there is no return statement the function returns *nothing* (None).

6.3 English Function Expressions (Phase 4)

A++ has built-in English-syntax function calls that read like prose:

A++ Expression	Python Equivalent	Result
absolute of -9	abs(-9)	9
square root of 144	math.sqrt(144)	12.0
ceiling of 4.1	math.ceil(4.1)	5
floor of 4.9	math.floor(4.9)	4
factorial of 6	math.factorial(6)	720
length of [1,2,3]	len([1,2,3])	3
sum of [1,2,3,4,5]	sum([1,2,3,4,5])	15
maximum of 17 and 42	max(17, 42)	42
minimum of 17 and 42	min(17, 42)	17
power of 2 and 10	pow(2, 10)	1024

6.4 Statistics and Random (Phase 5)

```
set data to [4, 8, 6, 5, 3, 2, 8, 9, 2, 5]

set avg  to mean of data           // average value

set med  to median of data         // middle value

set mod  to mode of data           // most frequent

say "Mean: {avg} Median: {med} Mode: {mod}"

set r    to random integer between 1 and 100

set rf   to random number between 0.0 and 1.0

set colours to ["red", "green", "blue"]

set pick to random choice from colours

say pick
```

7. Function Chaining (Phase 2)

A++ lets you chain operations with a dot `.`. Each step passes its result to the next. The final result is assigned to the target variable. This makes multi-step transformations read left-to-right like prose.

Syntax: `set VAR to EXPR. step1. step2. step3.`

```
// Read user input, strip whitespace, convert to integer:

ask raw "Enter age: "

set age to raw. trim. convert to int.


// Chain string transformations:

set name to " Anand ". trim. uppercase.


// Supported chain steps:

//   trim           remove leading/trailing whitespace

//   uppercase      convert to ALL CAPS

//   lowercase       convert to all lowercase

//   convert to INT  convert to any type (int, float, string, boolean)

//   call FUNCNAME   call a user-defined function with the value
```

■ **Tip:** Dots inside string literals and float numbers (like 3.14) are protected — they will never be treated as chain separators.

8. Error Handling

8.1 Try / Catch / Finally

Use `try` to wrap risky operations. If anything goes wrong inside the block, execution jumps to `catch`. The `finally` block always runs.

```
try

  ask raw "Enter a number: "

  set n to convert raw to int

  say "Your number doubled: {n * 2}"

catch err

  say "That was not a number! Error: {err}"

finally

  say "Thanks for using the calculator!"

end try
```

8.2 Assertions

Use `check ... or fail ...` to assert a condition is true. If the condition is false, the program stops with your message.

```
set score to 95

check score >= 0 or fail "Score cannot be negative"

check score <= 100 or fail "Score cannot exceed 100"

say "Score is valid: {score}"
```

9. Collections — Lists and Maps

9.1 Lists

```
set fruits to ["apple", "mango", "grape"]

append "kiwi" to fruits           // add to end

remove "apple" from fruits        // remove first match

set first_fruit to fruits[0]      // index access

set count to length of fruits     // number of items

// Iterate:

for each fruit in fruits:

    say fruit

end of for
```

9.2 Maps (Dictionaries)

```
set person to map

set person["name"] to "Anand"

set person["age"] to 25

set person["city"] to "Bengaluru"
```

```
say person["name"]           // Anand

say person                   // full map


// Check membership:

if "name" is in person then:

    say "Name is: {person['name']} "

end of if-else
```

10. Input and Output

10.1 Output — `say`

```
say "Hello, world!"

say 42

say true


set name to "Anand"

say "Hello, {name}!"           // f-string auto

say "Pi is approximately {3.14159}"
```

10.2 Input — `ask`

```
ask name "What is your name? "

say "Hello, {name}!"


ask age_str "How old are you? "

set age to convert age_str to int

say "You are {age} years old."
```

■ **Tip:** The `ask` command always stores the input as a string. Use `convert VAR to int` or chain `.convert to int.` when you need a number.

11. Standard Library

Import modules with `use MODULE`. The following modules are available. Math and statistics functions are available by default even without an explicit import.

use math

Functions: `sqrt`, `floor`, `ceiling`, `pi`, `sin`, `cos`, `tan`, `log`, `log10`, `abs`, `round_num`, `power`, `random`, `random_int`, `shuffle`, `choice`, `infinity`

```
use math

set r to sqrt(144)

say pi
```

use strings

Functions: `length`, `uppercase`, `lowercase`, `trim`, `split_text`, `join_text`, `replace_text`, `contains_text`, `starts_with`, `ends_with`, `to_number`, `to_integer`, `to_text`, `text_at`, `substring`

```
use strings

set s to "  hello  "

set clean to trim(s)
```

use lists

Functions: `count`, `sort_list`, `reverse_list`, `first`, `last`, `is_empty`, `sum_list`, `min_of`, `max_of`, `flatten`, `range_list`, `zip_lists`, `unique`

```
use lists

set s to sort_list([3, 1, 2])

say s
```

use io

Functions: `read_file`, `write_file`, `append_file`, `file_exists`, `print_line`, `get_input`, `clear_screen`

```
use io

set text to read_file("data.txt")

say text
```

use time

Functions: `wait`, `now`, `today`, `timestamp`

```
use time

say now()

say today()
```

Import everything at once:

```
use everything // loads all modules
```

12. IECE — Intelligent Error Correction Engine

The IECE is what makes A++ unique. It runs before the parser on every line and applies a pipeline of corrections to make the interpreter forgiving of natural human mistakes.

12.1 The IECE Pipeline

Step	What it does	Example
1. Normalize	Comments, memory recall, Python→A++, abbreviations	<code>print(x) → say x</code>
2. Split Merged	Detects accidentally fused keywords	<code>sayhello → say hello</code>
3. Strip Noise	Removes obvious filler words	<code>set the x to 5 → set x to 5</code>
4. Transpose	Corrects swapped word order	<code>x set to 5 → set x to 5</code>
5. Insert KW	Adds missing structural keywords	<code>set x 10 → set x to 10</code>
6. Fuzzy Correct	Similarity-matched keyword spelling	<code>repeaat → repeat</code>
7. Var Correct	Suggests close variable name matches	<code>scroe → score (if score is defined)</code>
8. Phrase Correct	Whole-line typo patterns	<code>insrease x bi 5 → increase x by 5</code>

12.2 IECE Modes

Mode	Command	Behaviour
Standard (default)	<code>mode standard</code>	Auto-fix obvious issues. Ask for ambiguous cases.
Auto	<code>mode auto</code>	Apply all corrections silently. No prompts. Best for files.
Learn	<code>mode learn</code>	Explain every correction. Always interactive. Great for learning.

12.3 Learning Memory

IECE stores corrections in a SQLite database ([iece_memory.db](#)). Corrections carry a confidence score (0–100). High-confidence (≥ 80) corrections are applied silently without prompting. Confidence decays if a correction is unused for 30 days.

```
memory    // show all learned corrections

forget    // wipe all corrections

sync      // force save to disk

stats     // show statistics

teach wrong correct  // manually add a correction

suggest   // show suggestions for the last failed line
```


13. Debugger

The built-in step-through debugger lets you pause execution, inspect variables, and walk through a program line by line.

13.1 Enabling the Debugger

```
debug on          // enable step-through mode

debug off         // disable

bp 15             // set breakpoint at line 15

bplist           // list breakpoints
```

13.2 Breakpoints in Code

```
set x to 10

breakpoint        // execution pauses here in the debugger

set x to x + 1

say x
```

13.3 Debugger Commands

Command	Action
<code>c / continue</code>	Continue to next breakpoint
<code>s / step / n</code>	Execute one line and pause again
<code>r / run</code>	Run without stepping (disable step mode)
<code>q / stop</code>	Stop execution completely
<code>env</code>	Print all current variables
<code>p EXPR</code>	Evaluate and print any expression
<code>bp N</code>	Set a new breakpoint at line N

14. Complete Program Examples

Each example below shows the same program in A++ and Python side-by-side so you can see how natural A++ reads.

14.1 Grading System

■ A++ (grades.aplus)

```
set scores to [85, 92, 71, 96, 60]

set total to sum of scores

set count to length of scores

set avg to total divided by count


say "Average score: {avg}"


if avg >= 90 then:

    say "Grade: A"

else if avg >= 75 then:

    say "Grade: B"

else if avg >= 60 then:

    say "Grade: C"

else:

    say "Grade: F"

end of if-else
```

■ Python (grades.py)

```
scores = [85, 92, 71, 96, 60]

total = sum(scores)

count = len(scores)

avg = total / count


print(f"Average score: {avg}")


if avg >= 90:

    print("Grade: A")

elif avg >= 75:

    print("Grade: B")

elif avg >= 60:

    print("Grade: C")

else:

    print("Grade: F")
```

14.2 Fibonacci Sequence

■ A++ (fibonacci.aplus)

```
define fibonacci taking n

  if n <= 1 then:

    return n

  end of if-else

  set a to fibonacci(n - 1)

  set b to fibonacci(n - 2)

  return a + b

end function

for each i in range(10):

  say fibonacci(i)

end of for
```

■ Python (fibonacci.py)

```
def fibonacci(n):

    if n <= 1:

        return n

    return fibonacci(n-1) \

        + fibonacci(n-2)

for i in range(10):

    print(fibonacci(i))
```

14.3 Simple Calculator

■ A++ (calculator.aplus)

```
say "Simple Calculator"

ask num1 "First number: "

ask op   "Operator (+,-,*,/): "

ask num2 "Second number: "


set a to convert num1 to float
set b to convert num2 to float


if op == "+" then:

    say a + b

else if op == "-" then:

    say a - b

else if op == "*" then:

    say a * b

else if op == "/" then:

    if b == 0 then:

        say "Cannot divide by zero"

    else:

        say a / b

    end of if-else

else:

    say "Unknown operator"

end of if-else
```

■ Python (calculator.py)

```
print("Simple Calculator")

num1 = input("First number: ")

op   = input("Operator: ")

num2 = input("Second number: ")


a = float(num1)
b = float(num2)


if op == "+":

    print(a + b)

elif op == "-":

    print(a - b)

elif op == "*":

    print(a * b)

elif op == "/":

    if b == 0:

        print("Divide by zero")

    else:

        print(a / b)

else:

    print("Unknown operator")
```

14.4 Statistics Analyser

■ A++ (stats.aplus)

```
set data to [12, 45, 7, 23, 89, 34, 56, 11, 78, 42]
```

```
set n to length of data
```

```
set avg to mean of data
```

```
set med to median of data
```

```
set mod to mode of data
```

```
set hi to maximum of max_of(data) and 0
```

```
set lo to minimum of min_of(data) and 999
```

```
say "Count : {n}"
```

```
say "Mean : {avg}"
```

```
say "Median : {med}"
```

```
say "Mode : {mod}"
```

```
say "Highest: {hi}"
```

```
say "Lowest : {lo}"
```

■ Python (stats.py)

```
import statistics as st
```

```
data = [12, 45, 7, 23, 89, 34, 56, 11, 78, 42]
```

```
n = len(data)
```

```
avg = st.mean(data)
```

```
med = st.median(data)
```

```
mod = st.mode(data)
```

```
hi = max(data)
```

```
lo = min(data)
```

```
print(f"Count : {n}")
```

```
print(f"Mean : {avg}")
```

```
print(f"Median : {med}")
```

```
print(f"Mode : {mod}")
```

```
print(f"Highest: {hi}")
```

```
print(f"Lowest : {lo}")
```

14.5 Multiplication Table

■ A++ (times_table.aplus)

```
ask n "Times table for: "  
  
set n to convert n to int  
  
say "--- {n} Times Table ---"  
  
for each i in range(1, 13):  
    set product to n * i  
    say "{n} x {i} = {product}"  
end of for
```

■ Python (times_table.py)

```
n = int(input("Times table for: "))  
  
print(f"--- {n} Times Table ---")  
  
for i in range(1, 13):  
    product = n * i  
    print(f"{n} x {i} = {product}")
```

15. REPL Command Reference

These commands are typed directly at the `A++[n]>` prompt. They control the interpreter session, not the program.

Command	Description
<code>help</code>	Show syntax reference
<code>ops</code>	Show full operator table
<code>env</code>	List all current variables and functions
<code>clear</code>	Reset environment (wipe all variables)
<code>undo</code>	Undo the last execution step
<code>redo</code>	Redo the last undone step
<code>memory</code>	Show all IECE learned corrections
<code>forget</code>	Wipe all IECE corrections
<code>sync</code>	Force-save IECE memory to disk
<code>stats</code>	Show IECE memory statistics
<code>suggest</code>	Show correction suggestions for the last error
<code>teach WRONG CORRECT</code>	Manually add a correction to IECE memory
<code>mode standard</code>	Set IECE to standard mode (default)
<code>mode auto</code>	Set IECE to silent auto-correct mode
<code>mode learn</code>	Set IECE to interactive learn mode
<code>debug on / debug off</code>	Toggle step-through debugger
<code>bp N</code>	Set breakpoint at line N
<code>bplist</code>	List all breakpoints
<code>use MODULE</code>	Import a standard library module
<code>exit / quit</code>	Exit the REPL

16. Complete Keyword Index

Keyword	Category	Purpose
set ... to	Assignment	Create or update a variable
increase ... by	Assignment	Add to a variable (+=)
decrease ... by	Assignment	Subtract from a variable (-=)
add ... to	Assignment	Add to variable
subtract ... from	Assignment	Subtract from variable
multiply ... by	Assignment	Multiply variable
divide ... by	Assignment	Divide variable
say	Output	Print a value
ask VAR PROMPT	Input	Read user input into VAR
if ... then:	Control	Conditional block (colon style)
if ... then	Control	Conditional block (classic style)
else if ... then:	Control	Additional condition branch
else:	Control	Default branch
end of if-else	Control	Close if/else block (colon style)
end if	Control	Close if/else block (classic)
while ...:	Control	While loop (colon style)
end of while	Control	Close while loop (colon style)
repeat N times	Control	Fixed-count loop
end loop	Control	Close repeat or while loop
for each VAR in	Control	Iterate over list or range
end of for	Control	Close for-each loop (colon style)
define ... taking	Functions	Define a function with parameters
end function	Functions	Close a function definition
return	Functions	Return a value from a function
call ... with	Functions	Call a function with arguments

<code>convert ... to</code>	Types	Convert a value to another type
<code>append ... to</code>	Lists	Add item to end of list
<code>remove ... from</code>	Lists	Remove item from list
<code>set X[k] to</code>	Maps	Set a map key
<code>try</code>	Errors	Begin a try block
<code>catch ERR</code>	Errors	Handle an exception
<code>finally</code>	Errors	Always-run block
<code>end try</code>	Errors	Close try block
<code>check ... or fail</code>	Errors	Assert a condition
<code>use MODULE</code>	Library	Import a standard library module
<code>import FILE</code>	Library	Import another .applus file
<code>breakpoint</code>	Debug	Pause execution in debugger
<code>delete VAR</code>	Misc	Delete a variable
<code>show type of VAR</code>	Misc	Print the type of a variable

17. Language Rules and Conventions

- **One statement per line.**

Each A++ line is one statement. There is no statement terminator.

- **Keywords are case-insensitive.**

SET, set, and Set all work. Only the first token is lowercased; variable names preserve your capitalisation.

- **Variable names are case-sensitive.**

Score and score are different variables.

- **Strings use double quotes.**

Use "double quotes" for strings. Single quotes also work but double quotes are standard.

- **Indentation is cosmetic inside files, required in REPL.**

In .aplus files, indentation is ignored — blocks are defined by their openers and closers. In the REPL, one tab per block level is inserted automatically when inside a block.

- **Comments start with // or #.**

Everything after // or # on a line is ignored.

- **Assignment and comparison are distinct.**

`set x to 5` assigns. `x == 5` compares. `is equal to` compares. There is no ambiguity.

- **English operators and symbol operators are identical.**

`is greater than` and `>` produce exactly the same code.

■ **Tip:** The IECE will often catch and fix rule violations automatically. If it asks 'Did you mean ...?' — press Enter or Y to accept. Press N to reject and retype the line manually.

About A++

A++ was designed and built by **Vishnu Theertha. A** in Bengaluru, Karnataka, India. The project started from a simple question: *"Why should beginners need to learn a foreign syntax before they can learn to think like a programmer?"*

A++ is released under the **MIT License** and is free and open-source. You are welcome to use, modify, and distribute it freely under the terms of that license.

Detail	Value
Author	Vishnu Theertha. A
Location	Bengaluru, Karnataka, India
Version	0.9 Prototype
Started	21 / 04 / 2026
Updated	23 / 04 / 2026
License	MIT License (SPDX: MIT)
Language	Python 3.10+ (no pip packages required)
GitHub	github.com/anandathirtha2009-art/aplus-language